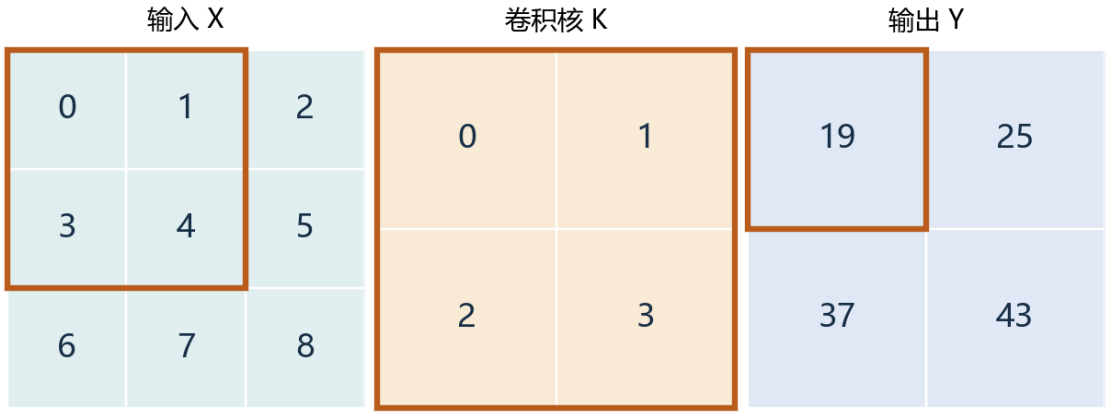
6.2 图像卷积

教材 PDF 240-246 · 正文 222-228

01 从共享规则到一个具体输出

6.1 已经把不同位置的权重约束成同一个核。现在要把这个规则真正算出来。将核放到一个窗口上，把对应位置的数相乘，再把乘积相加，就得到响应图中的一个元素。沿所有合法位置重复计算，得到整张特征图。



第一个窗口、完整核和对应的输出格用橙框连接。每一格都使用同一个核。

本例输入为 3×3、核为 2×2。左上窗口 [[0,1],[3,4]] 与核 [[0,1],[2,3]] 相乘相加得到 19。窗口右移一格得到 25，下移后得到 37、43。每一步都共享四个核权重。

为什么输出变小

核高为 h 时，窗口最后一个合法起点是 H-h。从 0 开始计数，共 H-h+1 个起点。宽度同理。本节只计算完整窗口，步幅为 1。输出形状因此是 (H-h+1,W-w+1)。

接下来要连起来的三件事

先把滑窗写成保留梯度的程序，再给核登记可训练参数，最后用目标响应计算损失并更新参数。检测边缘是贯穿全节的任务，感受野说明这些局部计算层叠后会覆盖多大区域。

内置 Conv2d 采用互相关约定，核保持当前顺序。数学卷积可通过双轴翻核对应。本例翻核后左上输出为 5，完整结果为 [[5,11],[23,29]]。

6.2 图像卷积

02 把每一个窗口明确写出来

PYTHON

```
import torch
from torch import nn
from torch.nn import functional as F

def corr2d(X, K):
    h, w = K.shape
    rows = X.shape[0] - h + 1
    cols = X.shape[1] - w + 1
    outputs = []
    for i in range(rows):
        line = []
        for j in range(cols):
            patch = X[i:i+h, j:j+w]
            value = (patch * K).sum()
            line.append(value)
        outputs.append(torch.stack(line))
    return torch.stack(outputs)
```

这里把 6.1 的核扫描拆成两层循环。i 选择输出行，j 选择输出列。窗口取自输入相同的左上起点。后面的完整教师文件还检查核是否非空、能否放入图像。

顺着数据走一遍

K.shape 拆出两个整数。range(rows) 从 0 遍历到 rows-1。切片右端不包含终点，因此 i:i+h 恰好取 h 行。

patch*K 是逐元素乘法，sum 把窗口内所有乘积加成一个标量。append 把标量放进一行列表，内层 stack 把这一行拼成张量，外层 stack 再把各行叠成二维响应。

怎样汇总成矩阵乘法

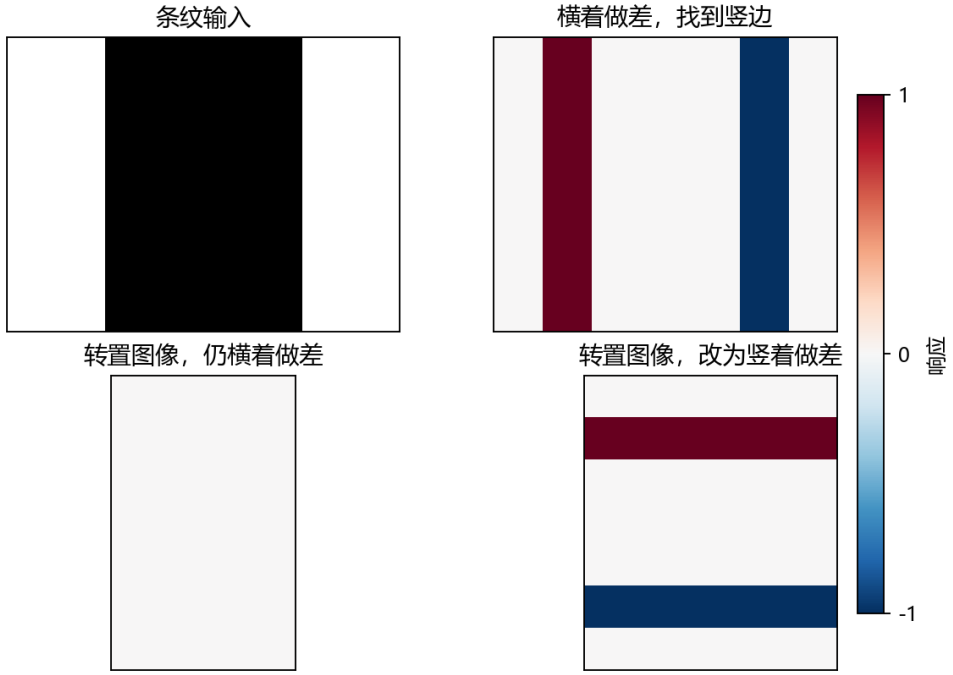
将每个窗口摊成一列，核摊成一行。第一列为 [0,1,3,4]，核行为 [0,1,2,3]。做 (1,4)@(4,4)，得到四个分数后排回 2×2。教师用 F.unfold 提取窗口并逐项核对，这对应教材题 3。

这种组织方式改变了计算的表达，核依然在所有窗口共享。

6.2 图像卷积

03 为什么相邻像素之差能找到边缘

如果窗口里两个像素相同，左减右得到 0。如果从亮处进入暗处，得到正值，反方向得到负值。于是核 [[1,-1]] 可以将水平方向的亮度变化变成明显响应。



四幅图对应原输入、正确差分、转置后方向不匹配，以及转置核后的响应。

条纹图为 6×8，中间四列为 0，其余为 1。输出为 6×7，首行 [0,1,0,0,0,-1,0]。非零值位于两条边界，符号表示亮度变化方向。

计算方向与边缘方向

水平做差读取左右像素，因此检测到沿竖直方向延伸的边缘。把图像转置，边缘改为水平延伸，继续左右做差得到全零。将核也转为 2×1、改做上减下，就重新得到边缘响应。

教材题 1 的斜边

用列号大于等于行号的像素组成一张 6×6 图。横向差分得到五个 -1。转置输入，或改用转置后的核，都会得到五个 +1。两张响应图互为转置。

斜边同时包含水平和垂直亮度变化。通过转置输入和核，可以分开观察计算方向改变了什么。

此处的核由人指定。下一步保留输入和目标响应，让程序从误差中学出这组权重。

6.2 图像卷积

教材 PDF 240-246 · 正文 222-228

04 将一个固定规则变成可训练层

PYTHON

```
class Conv2D(nn.Module):
    def __init__(self):
        super().__init__()
        self.weight = nn.Parameter(torch.rand(1, 2))
        self.bias = nn.Parameter(torch.zeros(()))

    def forward(self, X):
        return corr2d(X, self.weight) + self.bias
```

nn.Parameter 登记可训练张量，优化器可以通过 model.parameters() 找到它们。forward 使用这些参数进行乘加，偏置由全部位置共用。共享也意味着一个权重的梯度会汇总所有使用它的窗口。

给定目标 T，平方误差和 L=sum((Y-T)^2) 对某个输出的导数是 2*(Y-T)。再沿窗口乘法反传，核中某个权重的梯度就是各窗口误差导数乘对应输入像素，再跨位置求和。

一次训练怎样连起来

前向得到预测 Y，损失衡量预测与目标的差异。zero_grad 清除上次梯度，backward 计算本次梯度，step 按学习率更新参数。下一次前向使用更新后的核，重复这个过程才构成训练。

教师训练不使用偏置，只学习两个核权重。手写层与内置层复制同一初值，使用相同 SSE 和 SGD 0.03，比较每一步的参数与梯度。

损失尺度要跟着算

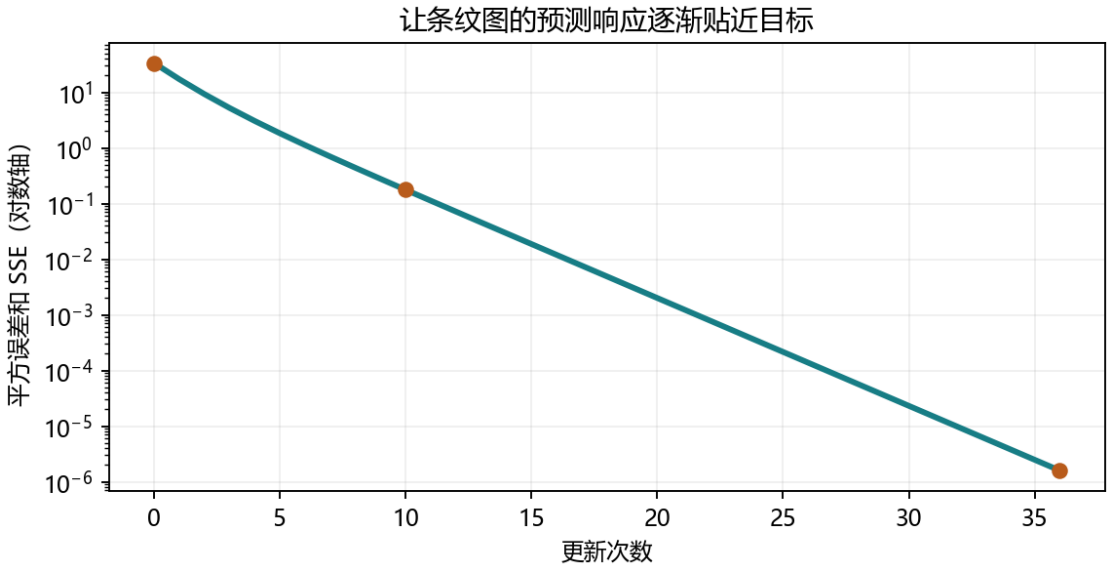
6×7 响应有 42 个元素。将平方误差 sum 改成 mean，会把损失和梯度都除以 42。这个无动量、同起点实验若要保持相同一步更新，mean 对应的学习率要乘以 42。

本节教师基准使用 SSE，最后的综合练习明确使用 MSE。每次比较先写清损失定义和学习率。

6.2 图像卷积

05 从训练曲线回到每一次参数更新

教师基准将 6×8 条纹图输入无偏置层，只学习两个核权重。目标由已知的 [1,-1] 核产生。两种实现复制相同初始核，用 SSE、SGD 0.03 执行 36 次更新，使比较集中在同一条训练路径上。



纵轴采用对数刻度，橙点标出初始、第 10 次和第 36 次更新后的 SSE。

初始 SSE 为 33.503179，第 10 次为 0.176467，第 36 次约为 1.59e-6。最终核约为 [1.00016,-0.99984]，因此重新计算响应时，已经能近似恢复目标的左右差分。

曲线在解释什么

每次更新改变同一组核权重，这组权重又同时改变所有窗口的输出。曲线将这些输出的误差统一汇总，使你能观察前向、反传和更新连续发生的效果。

损失下降后，还要检查学到的参数是否接近所需规则。教师再输入另一条灰度序列，响应最大误差约为 2.77e-4，核对同一局部规则在新像素值上是否仍适用。

实验条件怎样记录

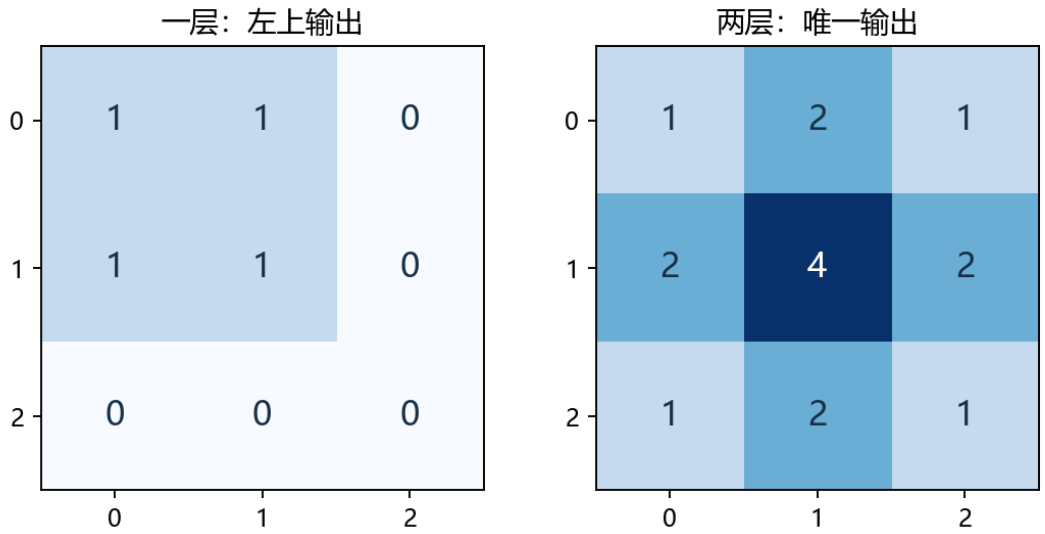
记录输入与目标、初值、损失定义、学习率和更新次数。曲线形状依赖这些条件，下面的跟做会明确指定初始核，便于单独改变损失与学习率。

学会一个局部规则后，还可以把多个卷积层串起来。下一页据此讨论，一个后层输出能受到原图哪些位置的影响。

6.2 图像卷积

06 从一层窗口走到多层感受野

第一层 2×2 核的一个输出只读取原图的 2×2 区域。第二层再读取第一层相邻四格，它们在原图上覆盖的区域合起来变成 3×3。某个输出可能依赖的输入范围称为感受野，全部位置的响应组成特征图。



全 1 核层叠后的输入梯度。中心像素通过四条路径影响最后一个输出。

较深的局部网络可以在保持每层小窗口的同时，组合更大范围的信息。第一层的边缘、角点等局部响应，成为后层进一步组合的输入。

梯度与结构覆盖的关系

两层核全为 1 时，最终输出对原始输入的梯度如下。

结果 / 结构

```
1 2 1
2 4 2
1 2 1
```

每个数字表示该像素经过多少条加权路径贡献到最后。

一般权重下，路径的贡献可以抵消。结构上的覆盖范围与某次实际梯度的大小需要分别观察。

教材题 4 的三个构造

二阶差分可用 [1,-2,1]，采样间隔为 h 时再除以 h 的平方。

固定宽度的离散积分可用全 1 核求和，再乘采样间隔。全局前缀积分可用累积和表达。

一维等间隔的 d 阶普通有限差分至少需要 d+1 个点，例如三阶差分 [-1,3,-3,1]。提高近似精度通常需要更大的窗口。

6.2 图像卷积

教材 PDF 240-246 · 正文 222-228

07 操作一 • 运行并读懂完整训练

在 VS Code 打开项目根目录，解释器选择已有 openmmlab，通过“终端 → 新建终端”打开 PowerShell。运行下面的中文导读模式。

结果 / 结构

```
$pythonPath = 'D:\Anaconda\envs\openmmlab\python.exe'
& $pythonPath lessons/ch06/02_image_convolution.py --guided
```

按控制台步骤，先核对手算输出，再看条纹响应，接着看 0、10、36 次更新的 SSE 与最终核，最后读两层感受野的梯度。打开教师文件，依次定位 corr2d、Conv2D、edge_demo、learn_kernel、receptive_field_demo。

需要重建基准记录和四图时，在命令末尾加 --record。训练图保存在 reports/ch06/figures/6_2_fit.png，指标保存在 reports/ch06/6_2_metrics.json。

每一组结果怎样核对

观察项	教师基准结果
滑窗输出	19、25、37、43
条纹首行	0、1、0、0、0、-1、0
第 10 次更新 SSE	约 0.176467
第 36 次更新 SSE	约 1.59e-6
最终核	约 [1.00016,-0.99984]

如果程序正常结束，这表示教师实现完成了内部核对。接下来的跟做用明确给定的初始核，方便你逐项改参数、复现因果关系。

基准随机种子为 6201，前面的演示也会消耗随机数。单独新建层时，仅设置相同种子还需核对参数初值是否相同。

6.2 图像卷积

08 操作二・复现手算和边缘响应

在 .cache 新建 6_2_follow_along.py，先复制第 2 页的导入和完整 corr2d 函数，再把下面代码接到函数后面。保存并按 F5，选择“运行当前 Python 文件”。

PYTHON

```
X = torch.arange(9.).reshape(3, 3)
K = torch.arange(4.).reshape(2, 2)
print("手写输出", corr2d(X, K).tolist())

stripe = torch.ones(6, 8, dtype=torch.float64)
stripe[:, 2:6] = 0
edge_kernel = torch.tensor([[1., -1.]], dtype=stripe.dtype)
target = corr2d(stripe, edge_kernel)
print("形状", tuple(target.shape))
print("第一行", target[0].tolist())
wrong = corr2d(stripe.T, edge_kernel)
matched = corr2d(stripe.T, edge_kernel.T)
print("方向不匹配时非零数", wrong.count_nonzero().item())
print("转置图和核后的形状", tuple(matched.shape))
```

先预测再运行

第一次输出应为 [[19,25],[37,43]]。stripe[:,2:6] 表示全部行、索引 2 到 5 的列，将这些像素改为 0。

条纹目标形状是 (6,7)，第一行应与第 7 页表格一致。转置输入后继续使用水平核，非零数量为 0。连核一起转置，输出形状为 (7,6)，与原目标互为转置。

用改变方向检查理解

先在纸上画出转置前后的两条边界，再解释为什么输出的高宽交换，以及差分方向怎样跟着核改变。

如果窗口与核形状不匹配，先打印 patch.shape 和 K.shape。逐元素相乘要求两者在这里都恰好对应一个完整窗口。

下一页在这个文件后面继续接训练代码，沿用 stripe、target 和已经导入的 nn、F。

6.2 图像卷积

09 操作三・用明确初值学回差分核

PYTHON

```
model = nn.Conv2d(1, 1, (1, 2), bias=False, dtype=stripe.dtype)
with torch.no_grad():
    model.weight.copy_(torch.tensor([[[[.2, .7]]]], dtype=stripe.dtype))
optimizer = torch.optim.SGD(model.parameters(), lr=.03)
x4, t4 = stripe[None, None], target[None, None]
for step in range(37):
    prediction = model(x4)
    loss = (prediction - t4).square().sum()
    if step in (0, 10, 36):
        print("更新次数与 SSE", step, loss.item())
    if step == 36:
        break
    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    optimizer.step()
print("最终核", model.weight.detach()[0, 0].tolist())
```

循环包含初始读数和第 36 次更新后的读数，所以迭代 37 次，但只执行 36 次 step。copy_ 在 no_grad 中设置初值，训练前向仍会建立计算图。

结果与教师基准怎样对应

跟做核初值明确为 [.2,.7]，初始 SSE 为 30.9，36 次更新后应小于 1e-4，核应接近 [1,-1]。教师基准采用另一组随机初值，因此各步损失按各自初值解释。

单独改变一项再比较

先将循环中的 sum 改为 mean，保留学习率 0.03，重新运行整个文件，观察收敛速度改变。然后将学习率设为 0.03×42，再从明确初值运行，比较每步参数与原 SSE 方案。

每次运行都会重新设置初始核，因此能够比较同起点的更新。记录你修改的损失、学习率、初始损失与最终损失，并解释梯度为什么按这个比例缩放。

修改后恢复 sum 和 0.03，即回到本页跟做基准。

6.2 图像卷积

教材 PDF 240-246 · 正文 222-228

10 操作四 • 排查梯度与扩展窗口

训练是否连通，可以在 loss.backward() 后、optimizer.step() 前暂停，查看 model.weight.grad。它应当是一个形状为 (1,1,1,2) 的有限张量。将某个结果取成 Python 数值，再用 torch.tensor 重新包装，会建立新的独立张量，原先到核的计算路径随之中断。

教材题 2 问逐格写入输出时的自动求导行为。本机 PyTorch 2.4.1 的逐格赋值会形成 CopySlices，仍能向核反传。教师程序实际比较了该行为与重新包装数值导致的报错。手写跟做使用 stack 组织已有张量，计算图保持连续。

在教师文件 receptive_field_demo 中，将断点放在计算 grad_two 之后。查看一层和两层对输入的梯度，沿原图画出这些输出分别覆盖哪些像素，再核对 [[1,2,1],[2,4,2],[1,2,1]]。

怎样把结果接回训练原理

预测依赖核，损失依赖预测，因此损失通过每个窗口向同一个核传回梯度。训练代码清梯度、反传、更新三步的顺序，使每一次更新只使用本次损失对应的梯度。

常见运行差异怎样检查

维数错误时，核对二维手写输入与四维内置输入的区别。方向错误时，检查核是 1×2 还是 2×1。损失变化异常时，核对 sum/mean、学习率和是否重新设置初值。

loss.item() 用于打印数值。用于训练的 loss 张量仍应通过 backward 反传。检查值与实际反传张量要能对应到同一次前向。

前面的训练已经建立 6.3 所需的窗口规则。接下来调整边界和步幅，窗口内的乘加与参数共享方式继续沿用。

6.2 图像卷积

11 综合练习与知识回收

打开 exercises/ch06/02_image_convolution.py。输入换为 8×6，中间四行为 0。第一阶段完成 TODO 1-4 的窗口行列、切片和乘加。第二阶段在 TODO 5 构造 2×1 上下差分核，生成目标响应。第三阶段完成 TODO 6-8 的平均平方误差、反传和参数更新。

结果 / 结构

```
$pythonPath = 'D:\Anaconda\envs\openmmlab\python.exe'
& $pythonPath exercises/ch06/02_image_convolution.py
```

练习给定学习率 0.9、24 次更新，使用 MSE。先预测目标形状和第一列，再记录训练前后的损失与最终核。每一阶段未填完时，程序会显示对应提示，完成该阶段后重新运行即可。

解释时沿同一条计算链复述。窗口输入与核怎样产生响应，全部响应怎样组成特征图。

再说明目标怎样形成损失，梯度怎样汇总到共享权重，更新后的权重怎样改变下一次预测。

集中反馈

项目	你的预测或实测
输出形状与第一列	_____
初始与最终 MSE	_____
最终核与差分方向	_____
两层覆盖的原图范围	_____

教材题 1 在第 3 页，题 2 在第 10 页，题 3 在第 2 页，题 4 在第 6 页。集中解释这些问题后，再进入填充和步幅。

页码依据用户提供的 Release 2.0.0 中文教材，作者为 Aston Zhang、Zachary C. Lipton、Mu Li、Alexander J. Smola。教师基准与复核保存在 reports/ch06/6_2_metrics.json 和 6_2_verification.md。教材 6.2 PyTorch 2.4.1